LectureLens Windows Edition — Hybrid Recording Architecture

Engineering Design Document Author: D&R AI Solutions Engineering **Date:** May 2026 **Version:** 1.0 **Status:** For approval

---

0. Executive Summary

This document defines the engineering design for **LectureLens Windows Edition** — a Windows-based classroom recording solution that runs in parallel with the existing Android Edition. Both editions coexist on the same backend infrastructure, with separate management surfaces in the admin portal and independent license tiers.

Architecture choice: Hybrid (local-first recording + on-demand LiveKit live-watch + background cloud upload). This avoids every failure mode observed in the Android-only LiveKit pipeline (Egress concurrency limits, network-latency-induced “Start signal not received” failures, recording dependence on cloud reachability).

Key product principles: 1. **Reliability first** — recording must succeed even when network/cloud is degraded 2. **Independent scaling** — every Windows PC operates autonomously; no shared cloud bottleneck 3. **Optional cloud features** — live-watch and cloud sync are conveniences, not dependencies 4. **Zero impact on Android Edition** — the existing Android admin portal, schema, endpoints, and recording flow remain untouched 5. **Per-device licensing** — separate SKU and pricing tiers for Android vs Windows

Deliverables: - Windows recorder application (.NET 8 Windows Service + OBS-based recording engine) - Admin portal “Windows” section (parallel to existing “Devices” / “Recordings”) - License management subsystem (key generation, activation, validation) - Backend extensions (Windows-specific endpoints, additive schema) - Documentation and installer

---

1. Architecture Overview

1.1 System Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        CAMPUS NETWORK (per classroom)                    │
│                                                                          │
│   Lumens VC-TR1     Sennheiser TCC2 + Extron DMP    Display Monitor      │
│        │                        │                          ▲            │
│        │ RTSP / USB             │ Dante (preferred) / USB  │ HDMI       │
│        ▼                        ▼                          │            │
│   ┌──────────────────────────────────────────────────────┐             │
│   │                   WINDOWS MINI PC                    │             │
│   │   (Intel NUC / Dell OptiPlex Micro / HP EliteDesk Mini)  │           │
│   │                                                      │             │
│   │   ┌──────────────────────────────────────────────┐   │             │
│   │   │       LectureLens Windows Recorder           │   │             │
│   │   │       (.NET 8 Windows Service)               │   │             │
│   │   │                                              │   │             │
│   │   │   ┌──────────────────────────────────┐       │   │             │
│   │   │   │ Recording Engine (OBS-based)     │       │   │             │
│   │   │   │ - Camera (RTSP / USB UVC)        │       │   │             │
│   │   │   │ - Mic (Dante / USB)              │       │   │             │
│   │   │   │ - Screen capture (Display Capture)│      │   │             │
│   │   │   │ - Hardware encode (Intel Quick Sync)│    │   │             │
│   │   │   └──────────────────────────────────┘       │   │             │
│   │   │                  │                           │   │             │
│   │   │                  ▼                           │   │             │
│   │   │   ┌──────────────────────────────────┐       │   │             │
│   │   │   │ Local Storage (SSD)              │       │   │             │
│   │   │   │ - Chunked MP4 (60-second segments)│  ← SOURCE OF TRUTH      │
│   │   │   │ - Always reliable, never network-│       │   │             │
│   │   │   │   dependent                      │       │   │             │
│   │   │   └──────────────────────────────────┘       │   │             │
│   │   │                  │                           │   │             │
│   │   │                  ▼                           │   │             │
│   │   │   ┌──────────────────────────────────┐       │   │             │
│   │   │   │ Background Upload Service        │       │   │             │
│   │   │   │ - Uploads chunks to Azure Blob   │       │   │             │
│   │   │   │ - Resumable on network drop      │       │   │             │
│   │   │   │ - Retry with exponential backoff │       │   │             │
│   │   │   └──────────────────────────────────┘       │   │             │
│   │   │                                              │   │             │
│   │   │   ┌──────────────────────────────────┐       │   │             │
│   │   │   │ LiveKit Publisher (ON-DEMAND)    │       │   │             │
│   │   │   │ - Activates only when admin clicks │     │   │             │
│   │   │   │   "Live View" in portal          │       │   │             │
│   │   │   │ - Independent from recording     │       │   │             │
│   │   │   │ - Failure does NOT affect recording │    │   │             │
│   │   │   └──────────────────────────────────┘       │   │             │
│   │   │                                              │   │             │
│   │   │   ┌──────────────────────────────────┐       │   │             │
│   │   │   │ Heartbeat & Command Loop         │       │   │             │
│   │   │   │ - Reports health every 30 sec    │       │   │             │
│   │   │   │ - Pulls remote commands          │       │   │             │
│   │   │   │ - License validation per heartbeat │     │   │             │
│   │   │   └──────────────────────────────────┘       │   │             │
│   │   └──────────────────────────────────────────────┘   │             │
│   │                                                      │             │
│   │   Display: HDMI to TV/Monitor (Mini PC drives display)  │           │
```

```
│   └──────────────────────────────────────────────────────────┘                │
└──────────────────────────────────────────────────────────────────────────────┘
                                          │
                                          │ Internet (background)
                                          ▼
                              ┌──────────────────────────────────┐
                              │       LectureLens Cloud          │
                              │                                  │
                              │  Backend API (Railway / Node.js) │
                              │  - Heartbeat receiver            │
                              │  - License validator             │
                              │  - Recording document creator    │
                              │  - Live-watch trigger            │
                              │                                  │
                              │  Azure Blob Storage              │
                              │  - Receives chunked uploads      │
                              │  - Final MP4 storage             │
                              │                                  │
                              │  LiveKit Cloud (optional path)   │
                              │  - For live-watch only           │
                              │                                  │
                              │  MongoDB Atlas                   │
                              │  - Recording metadata            │
                              │  - License records               │
                              │  - Device registry               │
                              │                                  │
                              │  Admin Portal (Vercel)           │
                              │  - Existing Android section      │
                              │    (UNTOUCHED)                   │
                              │  - NEW Windows section           │
                              │  - License management            │
                              │  - Recording playback (shared)   │
                              └──────────────────────────────────┘
```

1.2 Why Hybrid (Recap)

Architecture	Recording Reliability	Multi-Room Concurrent	Live Watch	Network Tolerance
LiveKit-only (current Android)	Cloud-dependent	Limited by Egress slots	✓ Built-in	Latency-sensitive
Legacy local-only (deprecated)	✓ Always works	✓ Independent per device	✗ Not available	✓ Tolerant
★ **Hybrid (this design)**	✓ Always works	✓ Independent	✓ On-demand	✓ Tolerant

The hybrid model gets every benefit and accepts only one cost: slightly more code complexity (separate recording vs live-watch paths).

---

2. Coexistence with Android Edition

2.1 Hard Isolation Guarantees

The Windows Edition introduces **zero changes** to:

Component	Status
Existing Android APK	❌ No changes
Android-specific backend endpoints	❌ No changes (e.g., `/api/classroom-recording/*`)
Existing schema fields	❌ No changes (only additive new fields)
Existing admin portal pages: Devices, Recordings, Booking	❌ No changes
LiveKit pipeline and Egress logic	❌ No changes
Existing OTA mechanism for Android	❌ No changes
Existing class scheduling flow	❌ No changes

All Windows additions are additive: - New routes under `/api/windows/*` namespace - New schema fields with Windows-specific names (or new collections entirely) - New admin portal pages under `/windows/*` route prefix - New license collection (separate from existing user/device collections)

2.2 Shared Components

Some components are naturally shared across both editions:

Component	Sharing Approach
User authentication (login, JWT)	Same auth — admin user can manage both editions
Recording playback (MP4 viewer)	Same component — recording document distinguishes platform via field
Class scheduling / room booking	Same — a “room” can be served by either Android TV or Windows PC
Cloud storage (Azure Blob)	Same — separate folder hierarchy by platform
MongoDB Atlas instance	Same DB — separate collections for Windows-specific data

2.3 Room-to-Device Mapping

A classroom can have: - **Android TV only** (existing flow, no change) - **Windows PC only** (new flow) - **Both** (Windows PC primary; Android TV optional secondary view)

Class scheduling layer is platform-agnostic. The class document points to a room number; whichever device(s) is registered to that room will pick it up.

---

3. Windows Recorder App Design

3.1 Tech Stack

Layer	Technology	Rationale
Operating system	Windows 11 IoT Enterprise LTSC 2024	10-year support, no auto-updates, kiosk mode capable
Service framework	.NET 8 Windows Service	Native Windows, auto-start on boot, robust process model
Recording engine	OBS Studio (embedded via obs-websocket plugin)	Battle-tested, supports all needed sources, hardware encode
Camera input	RTSP (preferred) via OBS Media Source	Lumens VC-TR1 native protocol; no USB driver dependence
Audio input	Dante Virtual Soundcard → OBS Audio Capture	Sennheiser TCC2 native protocol; channel selection deterministic
Screen capture	OBS Display Capture (DXGI Desktop Duplication)	Native Windows capture API; hardware-accelerated
Hardware encoder	Intel Quick Sync (HEVC + H.264)	Built into i5-13500T+; offloads CPU
Network transport (cloud upload)	HTTPS chunked upload to Azure Blob via .NET BlobClient	Native Azure SDK, handles retries
Live-watch transport	LiveKit Windows SDK (when activated)	Reuses existing LiveKit infrastructure
Auto-update	MSIX installer + Windows Update for Business	Standard Windows app distribution

3.2 Recording Engine Details

OBS Studio runs as a child process under the .NET Windows Service. The service controls OBS via the `obs-websocket` plugin (open-source, Apache-2.0 licensed).

Why OBS over building from scratch: - Already supports RTSP, Dante (via DirectShow), Display Capture, and hardware encoding - Mature scene management (background composite, picture-in-picture, multi-source) - 10+ years of community-validated reliability for sustained recording - Plugin ecosystem handles edge cases we'd otherwise build ourselves - Free (GPL) with permissive use of binaries

Recording configuration: - **Output container:** Fragmented MP4 (`fmp4`) — allows partial-file playback and resumable upload - **Video codec:** H.264 (Quick Sync), 1080p30 default, 4K available on i7+ hardware - **Audio codec:** AAC, 128 kbps stereo, sourced from Dante channel 1 (configurable) - **Bitrate:** 4 Mbps video + 128 kbps audio = ~3.1 GB per hour - **Chunking:** Output split every 60 seconds into separate `.mp4` segments - **Storage path:** `C:\LectureLens\Recordings\<class-id>\<timestamp>-<seq>.mp4`

Composite layout (default scene): - Background: Camera (Lumens) full-frame - Top-right overlay: Screen capture (PiP, 25% size) - Audio: Dante channel 1 (TCC2 mixed feed) - Optional overlays: timestamp, room ID, watermark

Multi-camera support (future): - OBS supports unlimited camera sources - Scene switching via WebSocket commands from .NET service - Use cases: front camera + back-of-room camera, document camera

3.3 Local Storage Management

Storage strategy: - Each class gets its own folder - Chunks written every 60 sec (independent files) - Class-end manifest file lists all chunks in order - Rolling cleanup: chunks deleted from local SSD only after successful cloud upload + 24-hour grace period

Disk space governor: - Service monitors free disk space every 60 sec - Alert threshold: < 50 GB free - Auto-cleanup threshold: < 20 GB free → delete oldest uploaded recordings (preserving manifest) - Hard stop threshold: < 5 GB free → refuse to start new recording (avoid runaway disk fill)

Crash recovery: - All chunks are individually playable (fragmented MP4) - If service crashes mid-recording, partial chunks are recoverable on next start - Chunk manifest is appended atomically (write-ahead log style)

3.4 Background Upload Service

Upload flow: 1. After each chunk is written (every 60 sec), upload task queued 2. Upload runs in separate thread pool (does not block recording) 3. Each chunk uploaded to Azure Blob via `BlockBlobClient.UploadAsync` (resumable) 4. On success: chunk marked uploaded in local SQLite metadata db 5. On failure: retry with exponential backoff (1s, 4s, 16s, 60s, max 5 min) 6. After retry exhausted: chunk stays in local queue, next upload cycle will retry 7. Network completely down for hours → chunks accumulate locally, upload resumes when network returns

Bandwidth governance: - Configurable upload rate limit (default: 80% of available bandwidth) - Throttle higher during off-peak hours (configurable schedule) - Per-class concurrent upload limit (default: 4 chunks parallel)

Upload progress reporting: - Heartbeat includes `upload.successCount`, `upload.failCount`, `upload.pendingCount` - Admin portal shows per-class upload status - Class marked "fully uploaded" only when all chunks confirmed in cloud

Final merge: - After last chunk uploaded, backend triggers `MergeChunks` operation (server-side) - Server uses `ffmpeg concat demuxer` to merge chunks into single `full.mp4` - Original chunks kept for 24 hours (rollback safety), then archived/deleted

3.5 LiveKit Live-Watch (On-Demand, Optional)

Activation flow: 1. Admin opens "Live View" in portal for a Room 2. Portal calls `POST /api/windows/devices/:id/live-watch/start` 3. Backend issues `start_live_watch` command to device via heartbeat response 4. PC publishes camera + audio to LiveKit room (in addition to local recording — does NOT replace) 5. Admin's browser subscribes to LiveKit room and views stream 6. When admin closes Live View: `stop_live_watch` command, PC stops publishing

Critical isolation: - Live-watch runs as **separate process** from recording engine - Live-watch failure (network drop, LiveKit cloud down, ICE failure) **does not affect recording** - Recording continues to local SSD regardless of live-watch state - The earlier "Start signal not received" Egress failures are eliminated — this isn't Egress, it's standard publish/subscribe

Resource impact: - Live-watch publish adds ~10-15% CPU (minor on i5+) - Bandwidth: ~2 Mbps outbound during live-watch session - Acceptable trade-off given on-demand activation

3.6 Heartbeat & Command Protocol

Endpoint: `POST /api/windows/devices/:deviceId/heartbeat`

Request body:

```
{
  "deviceId": "win_<uuid>",
  "appVersionCode": 100,
  "appVersionName": "1.0.0",
  "ipAddress": "10.20.16.x",
  "macAddress": "...",
  "isRecording": true,
  "currentClassId": "class_<id>",
  "license": {
    "key": "<license-key>",
    "checksum": "<hmac>"
  },
  "health": {
    "cpu": { "usagePercent": 35, "temperature": 65 },
    "ram": { "freeGB": 8.2, "totalGB": 16, "usedPercent": 49 },
    "disk": { "freeGB": 245, "totalGB": 512, "usedPercent": 52 },
    "network": { "uploadKbps": 5800, "downloadKbps": 12000 },
    "camera": { "ok": true, "name": "Lumens VC-TR1", "via": "rtsp" },
    "mic": { "ok": true, "name": "Dante Channel 1 (TCC2)", "via": "dante" },
    "screen": { "ok": true, "resolution": "1920x1080" },
    "recording": {
      "isRecording": true,
      "currentClassId": "...",
      "chunksWritten": 42,
      "chunksUploaded": 38,
      "chunksPending": 4,
      "audioLevelDbfs": -22.3,
      "videoFps": 30
    },
    "liveWatch": { "active": false, "viewerCount": 0 }
  }
}
```

Response (from backend):

```
{
  "schedule": [ /* upcoming classes */ ],
  "commands": [
    { "id": "...", "command": "start_recording", "params": { "classId": "..." } }
  ],
  "appUpdate": { "versionCode": 101, "downloadUrl": "https://..." },
  "license": { "valid": true, "expiresAt": "2027-05-01T00:00:00Z" },
  "serverTime": "..."
}
```

Heartbeat interval: 30 seconds (same as Android)

Commands supported: | Command | Purpose | |—|—| | `start_recording` | Force start recording for given class | | `stop_recording` | Force stop current recording | | `start_live_watch` | Begin LiveKit publishing for live preview | | `stop_live_watch` | End LiveKit publishing | | `restart_obs` | Restart OBS subprocess (without service restart) | | `restart_service` | Restart .NET service (recording resumes from current chunk) | | `restart_pc` | Reboot Windows PC | | `pull_logs` | Upload last N MB of logs to backend | | `capture_screenshot` | Capture and upload current display screenshot | | `update_config` | Update recording config (bitrate, fps, etc.) | | `validate_license` | Force re-validate license | | `clear_recordings` | Delete uploaded recordings older than X days |

3.7 Auto-Update Mechanism

- Service checks for updates via heartbeat response (`appUpdate` field)
- New MSIX downloaded in background to `C:\LectureLens\updates\`
- Install scheduled for next idle window (no active recording)
- Service restarts post-install
- Rollback supported if heartbeat fails post-update

---

4. Backend Architecture (Windows Additions, Isolated)

4.1 New Routes (Additive)

All under `/api/windows/*` namespace — completely separate from `/api/classroom-recording/*` (Android).

```
 POST   /api/windows/devices/register            → Register a new Windows PC
POST   /api/windows/devices/:id/heartbeat       → Receive heartbeat
GET    /api/windows/devices                     → List all Windows devices (admin)
GET    /api/windows/devices/:id                 → Get specific Windows device
DELETE /api/windows/devices/:id                 → Deregister Windows device

POST   /api/windows/devices/:id/command         → Issue remote command
GET    /api/windows/devices/:id/commands        → List recent commands
GET    /api/windows/devices/:id/logs            → Get uploaded logs
POST   /api/windows/devices/:id/logs            → Device uploads logs

POST   /api/windows/devices/:id/upload-chunk    → Upload recording chunk
POST   /api/windows/devices/:id/finalize-class  → Trigger merge of chunks

POST   /api/windows/devices/:id/live-watch/start  → Begin live publish
POST   /api/windows/devices/:id/live-watch/stop   → End live publish
GET    /api/windows/devices/:id/live-watch/token  → Generate viewer token

POST   /api/windows/licenses/issue              → Issue new license (admin)
GET    /api/windows/licenses                    → List licenses (admin)
POST   /api/windows/licenses/:key/activate      → Activate license on device
POST   /api/windows/licenses/:key/revoke        → Revoke a license
GET    /api/windows/licenses/:key/validate      → Validate license

POST   /api/windows/app/upload                  → Upload new MSIX (admin OTA)
GET    /api/windows/app/download                → Device downloads APK/MSIX
GET    /api/windows/app/versions                → List available versions
```

4.2 New Schema (Additive)

Collection: `windows_devices` (separate from `lcs_classroomdevices`)

```
{
  _id: ObjectId,
  deviceId: "win_<uuid>",
  authToken: "<random-256-bit>",
  name: "Room 014 Windows PC",
  roomNumber: "014",
  spaceCode: "C25-BA-FG-R014",
  hardwareModel: "Intel NUC 13 Pro",
  osVersion: "Windows 11 IoT Enterprise LTSC",
  appVersionName: "1.0.0",
  appVersionCode: 100,
  ipAddress: "10.20.16.85",
  macAddress: "...",
  lastHeartbeat: Date,
  isOnline: Boolean,
  isRecording: Boolean,
  currentClassId: ObjectId,
  licenseKey: "WIN-XXXX-XXXX-XXXX",
  licenseExpiresAt: Date,
  health: { /* nested */ },
  alerts: [{ type, message, time }],
  createdAt: Date,
  updatedAt: Date
}
```

Collection: `windows_recordings` (separate from `lcs_recordings`)

```
{
  _id: ObjectId,
  scheduledClass: ObjectId,
  windowsDevice: ObjectId,
  pipeline: "windows-hybrid",

  recordingStart: Date,
  recordingEnd: Date,
  duration: Number,

  // Local-first chunks
  chunks: [{
    seq: Number,
    filename: "chunk-0042.mp4",
    sizeBytes: Number,
    durationMs: Number,
    azureBlobUrl: String,
    uploadedAt: Date,
    uploadStatus: "pending" | "uploading" | "uploaded" | "failed"
  }],

  // Final merged
  mergedVideoUrl: String,
  mergedFileSize: Number,
  mergeStatus: "pending" | "merging" | "ready" | "failed",
  mergeError: String,
  mergedAt: Date,

  // Live-watch (if used during this recording)
  liveWatchSessions: [{
    startedAt: Date,
    endedAt: Date,
    viewerCount: Number,
    livekitRoomName: String
  }],

  status: "recording" | "uploading" | "merging" | "completed" | "failed",
  isPublished: Boolean,

  createdAt: Date,
  updatedAt: Date
}
```

Collection: `windows_licenses` (new)

```
{
  _id: ObjectId,
  licenseKey: "WIN-XXXX-XXXX-XXXX-XXXX",
  tier: "starter" | "professional" | "enterprise",
  customerName: String,
  customerEmail: String,

  issuedAt: Date,
  activatedAt: Date,
  expiresAt: Date,

  // Device binding (one license = one device)
  boundDeviceId: ObjectId,
  boundAt: Date,
  hardwareFingerprint: String,

  // Features enabled (per-tier)
  features: {
    maxRecordingHoursPerDay: Number,    // null = unlimited
    liveWatchEnabled: Boolean,
    multiCameraEnabled: Boolean,
    fourKEnabled: Boolean,
    customOverlayEnabled: Boolean,
    cloudUploadEnabled: Boolean,
    apiAccessEnabled: Boolean
  },

  // Validity tracking
  status: "issued" | "active" | "expired" | "revoked" | "suspended",
  lastValidatedAt: Date,

  // Audit
  notes: String,
  issuedBy: ObjectId,
  revokedBy: ObjectId,
  revokedAt: Date,
  revokeReason: String
}
```

No changes to existing collections: - `lcs_classroomdevices` (Android devices) — UNTOUCHED - `lcs_recordings` (Android recordings) — UNTOUCHED - `lcs_scheduledclasses` (shared, no schema change) - `lcs_users` (shared) - `lcs_devicecommands` (Android commands; Windows uses separate collection `windows_devicecommands`)

4.3 Recording Document Differentiation

When admin views recordings, the system fetches from **both** `lcs_recordings` and `windows_recordings` , normalizes, and presents in unified Recordings page. The platform field distinguishes: - `pipeline: "livekit"` → Android (existing) - `pipeline: "local-windows"` → Windows (new)

Playback URL is the same field (`mergedVideoUrl` or `videoUrl`) — both serve MP4 from Azure Blob.

4.4 Backend Code Organization

```
backend/
├── routes/
│   ├── classroomRecording.js      ← UNTOUCHED (Android)
│   ├── recordings.js              ← Existing playback (works for both)
│   ├── classes.js                 ← Existing scheduling (platform-agnostic)
│   ├── auth.js                    ← Existing
│   ├── ...
│   └── windows/                   ← NEW directory
│       ├── devices.js             ← Windows device routes
│       ├── recordings.js          ← Windows recording routes
│       ├── licenses.js            ← License management
│       ├── liveWatch.js           ← Live-watch endpoints
│       └── appUpdate.js           ← Windows app OTA
│
├── controllers/
│   ├── classroomRecordingController.js  ← UNTOUCHED
│   └── windows/                          ← NEW directory
│       ├── deviceController.js
│       ├── recordingController.js
│       ├── licenseController.js
│       ├── liveWatchController.js
│       └── appUpdateController.js
│
├── models/
│   ├── ClassroomDevice.js          ← UNTOUCHED
│   ├── Recording.js                ← UNTOUCHED
│   ├── ScheduledClass.js           ← UNTOUCHED
│   └── windows/                    ← NEW directory
│       ├── WindowsDevice.js
│       ├── WindowsRecording.js
│       ├── WindowsLicense.js
│       └── WindowsDeviceCommand.js
│
└── middleware/
    ├── deviceAuth.js               ← UNTOUCHED (Android)
    └── windowsDeviceAuth.js        ← NEW (different auth header pattern)
```

Hard rule for code review: Any PR that modifies a file outside `backend/routes/windows/`, `backend/controllers/windows/`, or `backend/models/windows/` must be flagged as touching shared code and explicitly reviewed.

---

5. Admin Portal Design (New Windows Section)

5.1 Navigation Structure

```
LectureLens Admin Portal
├── Dashboard                          ← UNTOUCHED (shows both Android + Windows aggregate
│
├── Android (existing section)         ← UNTOUCHED — entire subtree
│   ├── Devices                        ← UNTOUCHED
│   ├── Recordings                     ← UNTOUCHED
│   ├── Booking                        ← UNTOUCHED
│   ├── App Update                     ← UNTOUCHED
│   └── ...
│
├── Windows (NEW section)              ← NEW
│   ├── Devices                        ← Windows PC fleet view
│   ├── Recordings                     ← Windows recordings (filtered view)
│   ├── Live View                      ← Live-watch interface
│   ├── App Update                     ← MSIX OTA management
│   └── License Management             ← Issue/revoke/track licenses
│
├── Facility (shared)                  ← UNTOUCHED (rooms can be either platform)
├── Users (shared)                     ← UNTOUCHED
├── Analytics                          ← UNTOUCHED (existing) + new Windows metrics tab
└── Settings                           ← UNTOUCHED + new Windows licensing settings
```

5.2 New Pages

Windows → Devices - List of all Windows PCs across campus - Per-device card: name, room, online/offline, app version, license status, recording state - Health metrics: CPU, RAM, disk, recording progress - Actions: View details, Send command, Live View, Revoke license - Filters: by license tier, license status (active/expiring/expired), online status

Windows → Recordings - List of recordings made by Windows devices - Same playback UI as Android (shared component) - Per-recording: chunks uploaded count, merge status, total size, duration - Distinguishes pipeline visually (badge: “Windows Hybrid”) - Combined view available: “All Recordings (Android + Windows)”

Windows → Live View - Grid of all online Windows devices - Click any device → live stream starts (LiveKit publish triggered on-demand) - Side panel: device health metrics - Multi-device view: split screen for up to 4 rooms simultaneously

Windows → App Update - Upload new MSIX file - Per-version metadata: version code, name, release notes, file size - Activate / deactivate versions - Per-device update status (which version each PC is running) - Rollback button (revert to previous version)

Windows → License Management - Issue new license: form with customer name, email, tier, expiry, features - License key auto-generated (cryptographically random with HMAC checksum) - List view: all issued licenses with status, expiry, bound device - Per-license actions: View details, Revoke, Extend, Transfer - Bulk operations: import customer list, generate batch licenses - Expiring soon alerts: licenses expiring in next 30 days - Revoked licenses: audit trail with reason - Export: CSV download for accounting

5.3 Frontend Code Organization

```
admin-portal/src/
├── pages/
│   ├── android/                    ← Renamed namespace (was top-level)
│   │                                  Wait — to avoid breaking existing,
│   │                                  KEEP existing pages at top-level
│   ├── Dashboard.jsx               ← Existing
│   ├── Devices.jsx                 ← Existing (Android) — UNTOUCHED
│   ├── Recordings.jsx              ← Existing — UNTOUCHED
│   ├── ...                         ← All existing pages UNTOUCHED
│   │
│   └── windows/                    ← NEW directory
│       ├── WindowsDevices.jsx
│       ├── WindowsRecordings.jsx
│       ├── WindowsLiveView.jsx
│       ├── WindowsAppUpdate.jsx
│       └── WindowsLicenseManagement.jsx
│
├── components/
│   ├── DeviceCard.jsx              ← Existing (Android-specific)
│   └── windows/                    ← NEW directory
│       ├── WindowsDeviceCard.jsx
│       ├── LicenseKeyDisplay.jsx
│       ├── LicenseTierBadge.jsx
│       └── LiveWatchPlayer.jsx
│
├── api/
│   ├── android.js                  ← Existing API client
│   └── windows.js                  ← NEW API client (separate)
│
└── routes/
    └── App.jsx                     ← Add new routes under /windows/*
```

Routing additions in App.jsx:

```
// Existing routes UNTOUCHED
<Route path="/dashboard" element={<Dashboard />} />
<Route path="/devices" element={<Devices />} />        // Android
<Route path="/recordings" element={<Recordings />} />   // Combined
// ... all existing untouched

// NEW Windows routes
<Route path="/windows/devices" element={<WindowsDevices />} />
<Route path="/windows/recordings" element={<WindowsRecordings />} />
<Route path="/windows/live-view" element={<WindowsLiveView />} />
<Route path="/windows/app-update" element={<WindowsAppUpdate />} />
<Route path="/windows/licenses" element={<WindowsLicenseManagement />} />
```

5.4 Sidebar / Top Nav

Add a “Windows” section to the existing sidebar — separate group, clearly labeled. Existing “Devices”, “Recordings”, etc. remain in their current positions.

```
Sidebar:
──────────────────────
Dashboard
──────────────────────
ANDROID
  • Devices
  • Recordings
  • Booking
  • App Update
──────────────────────
WINDOWS  ⭐ NEW
  • Devices
  • Recordings
  • Live View
  • App Update
  • Licenses
──────────────────────
SHARED
  • Facility
  • Users
  • Analytics
  • Settings
──────────────────────
Logout
```

This makes the platform separation immediately visible to admins.

---

6. License Management System

6.1 License Key Format

Format: `WIN-XXXX-XXXX-XXXX-XXXX` (5 groups of 4 alphanumeric chars)

Example: `WIN-7A3K-9X2P-NLM4-Q8RY`

Generation: - 16 random bytes from `crypto.randomBytes` - Encoded as base32 (uppercase alphanumeric, no ambiguous chars: removed I, O, 0, 1) - Split into 4-char groups - HMAC-SHA256 checksum with server secret appended internally (for tamper detection)

Anti-tampering: - Each license key has an embedded HMAC checksum (server-side validation) - Key cannot be modified locally — backend validation is authoritative - License document includes hardware fingerprint (binds key to specific PC)

6.2 License Tiers

Three tiers — pricing TBD by sales:

Feature			
Recording (local + cloud upload)	✓	✓	✓
Max recording hours/day	4 hours	12 hours	Unlimited
Resolution	720p	1080p	4K
Live-watch	✗	✓	✓
Multi-camera	✗	✗	✓
Custom overlays / branding	✗	✗	✓
API access	✗	✓	✓
Concurrent live viewers	—	5	Unlimited
Cloud storage included	100 GB	500 GB	Unlimited
Support	Email	Email + Phone	Dedicated rep

Tier features enforced at: - **Backend:** validates license tier on every heartbeat, returns `features` flags - **Windows app:** disables features not licensed (UI button greyed out, action returns "license required" error) - **Admin portal:** doesn't display features the customer's license doesn't include (clean UX)

6.3 Activation Flow

On first install: 1. Windows installer prompts for license key 2. Key sent to backend `/api/windows/licenses/<key>/activate` along with hardware fingerprint 3. Backend validates key (not revoked, not expired, not already bound to another device) 4. Backend records device binding 5. Returns activation token + features list 6. Windows app stores activation token in encrypted DPAPI store

On subsequent boots: 1. App reads activation token from local store 2. Sends to backend in heartbeat for re-validation 3. Backend checks: license still valid? Bound to this device? Expiry not passed? 4. Returns OK + updated features (in case admin changed tier) 5. If invalid: app enters "limited mode" (continues recording for 7-day grace period, shows warning)

Grace period: - License expired or backend unreachable for > 7 days → recording disabled (warning shown) - Recording resumes immediately on license renewal - Local recordings during grace period stay on disk and can be uploaded once license active

6.4 License Lifecycle

```
[Issued]    ──admin issues key──→ key generated, no device bound yet
  │
  ├─activate→  [Active]    ──renewal/extension→  [Active] (ongoing)
  │              │
  │              ├─expiry→  [Expired]   ──renew→  [Active]
  │              │            │
  │              │            └─grace 30d──→  [Suspended]
  │              │
  │              └─admin revoke→  [Revoked]  (terminal)
  │
  └─not activated within 90d→  [Expired (unused)]  (terminal)
```

6.5 Pricing Model Suggestions (For Sales)

Per-device, per-year pricing:

Tier	Suggested Price (INR/year)	Suggested Price (USD/year)
Starter (Android)	₹15,000	$200
Starter (Windows)	₹20,000	$275
Professional (Android)	₹35,000	$475
Professional (Windows)	₹45,000	$600
Enterprise (Windows)	₹75,000	$1,000

Rationale for Windows premium: - Better hardware utilization (4K, multi-camera, live overlays exclusive) - More reliable architecture (local-first) - Higher cloud bandwidth used (better quality) - Premium support tier

Volume discounts: - 10–49 devices: 10% off - 50–199 devices: 15% off - 200+: contact sales (custom)

Multi-year commitment: - 2-year prepaid: 8% off - 3-year prepaid: 15% off

These are starting points — final pricing should be calibrated against competitive products (Panopto, Echo360, Mediasite) which run $500–$2,500 per device per year.

---

7. Implementation Roadmap

Phase 1 — Foundation (Weeks 1–4)

- Set up Windows dev environment, OBS embedding research
- Build .NET 8 Windows Service skeleton with auto-start
- Implement OBS subprocess management (start, stop, configure via WebSocket)
- Recording engine: single-source recording to local SSD
- Disk space governor + chunk management
- **Milestone:** PC records local MP4 of camera feed continuously

Phase 2 — Cloud Integration (Weeks 5–8)

- Background upload service (Azure Blob)
- Resumable chunked upload with retry logic
- SQLite metadata DB (local upload tracking)
- Backend: new `/api/windows/devices/*` routes
- Backend: `WindowsDevice`, `WindowsRecording` schemas
- Heartbeat protocol implementation
- **Milestone:** PC heartbeats to backend, uploads complete recordings to Azure

Phase 3 — Class Lifecycle (Weeks 9–10)

- Schedule polling on Windows side
- Class start/stop logic (auto + force commands)
- Recording chunk merge on backend (ffmpeg concat)
- Recording status tracking and admin visibility

- **Milestone:** End-to-end class recording: schedule → record → upload → merge → playable in admin portal

Phase 4 — Admin Portal Windows Section (Weeks 11–13)

- New `/windows/devices` page (fleet view)
- New `/windows/recordings` page (Windows-only filter)
- New `/windows/app-update` page (MSIX management)
- Sidebar updates (Windows section)
- API client for windows endpoints
- **Milestone:** Admin can manage Windows fleet from portal

Phase 5 — License System (Weeks 14–16)

- License model + key generation
- Activation endpoints
- Hardware fingerprinting (Windows-side)
- License validation on heartbeat
- Admin portal license management page
- Tier enforcement (recording hours cap, resolution limits, etc.)
- **Milestone:** New device install requires license activation; expired licenses disable recording

Phase 6 — Live-Watch (Weeks 17–18)

- LiveKit Windows SDK integration
- On-demand publish trigger
- Admin portal live view UI
- Multi-room concurrent live view
- Tier-based viewer count limits
- **Milestone:** Admin can live-view any Windows PC's stream on demand

Phase 7 — Installer & Auto-Update (Weeks 19–20)

- MSIX packaging
- Windows installer with license prompt
- Auto-update mechanism via heartbeat
- Rollback on failure detection
- Documentation: installation guide
- **Milestone:** Single-click installer for new deployments

Phase 8 — QA & Pilot (Weeks 21–24)

- Internal QA: 5-room pilot deployment
- Stress testing: 50 concurrent Windows recordings
- Performance benchmarking vs Android
- Documentation: admin guide, troubleshooting guide
- Customer-facing release notes
- **Milestone:** Production-ready v1.0

Total timeline: ~6 months for first production release.

---

8. Tech Stack Decisions Summary

Decision	Choice	Alternative Considered	Reason
OS	Windows 11 IoT Enterprise LTSC	Standard Windows 11 Pro	LTSC = no auto-updates interrupting recording
Service framework	.NET 8 Windows Service	Node.js	Native Windows, better service hosting
Recording engine	OBS Studio (embedded)	Custom Media Foundation	OBS handles 95% of edge cases for free
OBS control	obs-websocket plugin	OBS-WebSocket-Js	Standard, mature
Camera transport	RTSP	UVC over USB	Eliminates USB driver issues; native to Lumens VC-TR1
Audio transport	Dante (DVS)	USB Audio	Industry standard for pro audio; avoids USB enumeration issues
Encoder	Intel Quick Sync	NVENC, software x264	Built into Mini PC CPU; no GPU needed
Local DB	SQLite	LiteDB	Battle-tested, single-file, embedded
Cloud storage	Azure Blob	AWS S3	Already in use for Android Edition
Backend	Existing Node.js	Separate microservice	Reuse infrastructure; isolated routes prevent code crossover
Live-watch	LiveKit Windows SDK	WebRTC custom	Reuse Android infrastructure
Installer	MSIX	MSI / EXE	Modern Windows packaging; auto-update support
License protection	HMAC-SHA256	RSA signature	Simpler, sufficient for non-DRM use
Hardware fingerprint	TPM 2.0 + machine GUID	MAC address only	Robust against hardware swaps

9. Hardware Requirements

Minimum spec (Starter tier): - Intel Core i3-13100T (4 cores) - 8 GB DDR4 RAM - 256 GB NVMe SSD - Gigabit Ethernet - Intel UHD Graphics (Quick Sync H.264)

Recommended spec (Professional / Enterprise): - Intel Core i5-13500T (12 cores) - 16 GB DDR4 RAM - 512 GB NVMe SSD - Gigabit Ethernet (dual NIC for AV VLAN + general) - Intel Iris Xe (Quick Sync H.264 + H.265 + AV1)

Form factor: - Mini PC (NUC, OptiPlex Micro, EliteDesk Mini) - Compact (~700g, fits behind monitor or under desk) - Active cooling (designed for sustained workload)

Reference SKUs: - Intel NUC 13 Pro NUC13ANHi5 - Dell OptiPlex 3000 Micro / 7010 Micro - HP EliteDesk 800 G9 Mini - ASUS PN64 / PN54

---

10. Security Considerations

10.1 Recording Data Protection

- **At rest (local):** BitLocker disk encryption enabled by default
- **At rest (cloud):** Azure Blob encryption (AES-256, Microsoft-managed keys)
- **In transit:** HTTPS/TLS 1.3 for all network traffic
- **Local file permissions:** Service runs as restricted local account; files inaccessible to interactive users

10.2 License Protection

- License keys stored in Windows DPAPI (encrypted with machine-bound key)
- Hardware fingerprint includes TPM 2.0 endorsement key (where available)
- Tampered local validation falls back to backend validation
- Heartbeat-based revocation: revoked license disables recording within ~30 sec

10.3 API Security

- JWT-based auth for admin portal users (existing system)
- Device-specific auth tokens for Windows PC API calls
- Rate limiting on all license endpoints (anti-brute-force)
- Audit log of all license operations

10.4 Update Channel Security

- MSIX packages signed with code-signing certificate
- Hash verification on download
- Rollback if post-update heartbeat fails

---

11. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
OBS subprocess crashes mid-recording	Medium	Recording gap	Service auto-restarts OBS, recovers from last chunk; chunks already on disk are safe
Windows Update interrupts class	Low	Recording lost	LTSC has no auto-updates; scheduled maintenance windows for security patches
Hardware failure (SSD, power)	Low	Recording lost	UPS (₹3K), local recording resumes from chunks already on disk
License validation backend unreachable	Low	Recording disabled after grace	7-day grace period; recording continues, alerts shown
License key reverse-engineering	Low	Revenue loss	Backend validation is authoritative; local validation is just UX
Codebase divergence (Android vs Windows handlers)	Medium	Maintenance overhead	Separate directories enforce isolation; shared code only in models
OBS plugin version incompatibility	Medium	Service starts in degraded state	Pin OBS version; test new versions in pilot before fleet rollout
Customer migrates from Android to Windows mid-license	Medium	License confusion	Issue separate license keys; cross-grade discount policy
Windows app file permissions issues	Low	Service can't write recordings	Installer configures perms; health check in heartbeat

12. Migration & Rollout Plan

12.1 Internal Pilot

- Deploy to 1 lab Windows Mini PC
- Run for 1 week alongside existing Android setup in same room
- Validate recording quality, upload reliability, live-watch latency

12.2 Customer Pilot

- Onboard 1–2 friendly customers willing to test
- Provide 5 free Windows licenses for pilot (3-month duration)
- Weekly check-ins, gather feedback
- Parallel-run with their existing Android Edition for comparison

12.3 General Availability

- After pilot success criteria met:
- 99.5% recording success rate over 30 days
- <5 min average admin issue resolution time
- Customer Net Promoter Score ≥ 8

- Begin sales motion
- Documentation, video tutorials, sales collateral

12.4 Customer Migration Options

For existing Android customers who want Windows: - **Add-on:** Keep Android Edition, add Windows for new rooms - **Replace:** Phase out Android over 6–12 months as TV hardware reaches end-of-life - **Hybrid:** Some rooms Android, some Windows (admin portal handles both) - **Cross-grade discount:** Existing Android customers get 20% off Windows licenses for first year

---

13. Cost & Revenue Model

13.1 Customer-Facing Costs

Per Windows-based classroom (one-time + annual):

Item	One-time	Annual
Windows Mini PC (recommended spec)	₹50,000	—
Windows 11 IoT Enterprise LTSC license	₹12,000	—
UPS (1 KVA)	₹4,500	—
Wireless KB+Mouse	₹1,500	—
LectureLens Windows License (Professional tier)	—	₹45,000
Cloud storage (included in license)	—	included
Support (included in license)	—	included
Total per room	**₹68,000**	**₹45,000**

For comparison, Android Edition:

Item	One-time	Annual
LG 55TR3DK signage TV	₹85,000	—
LectureLens Android License (Professional tier)	—	₹35,000
Total per room	**₹85,000**	**₹35,000**

5-year TCO at 50 rooms: - Android: 50 × (85,000 + 5 × 35,000) = ₹1.30 Cr - Windows: 50 × (68,000 + 5 × 45,000) = ₹1.46 Cr

(Hardware refresh assumed every 5 years; license is annual subscription.)

The Windows Edition is positioned as the **premium tier** with features Android can't match — multi-camera, 4K, live overlays, better reliability — justifying the higher recurring cost.

13.2 D&R AI Solutions Revenue Model

Per-device-year revenue (estimated): - Android Starter @ ₹15,000: ~30% margin = ₹4,500/device/year - Android Professional @ ₹35,000: ~40% margin = ₹14,000/device/year - Windows Professional @ ₹45,000: ~45%

margin = ₹20,250/device/year - Windows Enterprise @ ₹75,000: ~50% margin = ₹37,500/device/year

Scaling projection at 200 devices (mixed): - 100 Android Pro × ₹14,000 = ₹14 lakh - 80 Windows Pro × ₹20,250 = ₹16.2 lakh - 20 Windows Enterprise × ₹37,500 = ₹7.5 lakh - **Total annual recurring revenue:** ₹37.7 lakh

---

14. Decision Points for Approval

Before implementation begins, please confirm:

#	Decision	Default	Approval
1	Hybrid architecture (local + on-demand LiveKit + cloud upload)	✓ Recommended	☐
2	Tech stack: .NET 8 + OBS Studio + Quick Sync	✓ Recommended	☐
3	License tiers: Starter / Professional / Enterprise	✓ Recommended	☐
4	Pricing: Windows premium (~30% over Android tier)	✓ Suggested	☐
5	Hard isolation from Android codebase	✓ Mandatory	☐
6	New admin portal section under “Windows”	✓ Recommended	☐
7	License validation on every heartbeat (with 7-day grace)	✓ Recommended	☐
8	6-month timeline to v1.0 production	✓ Estimated	☐

---

15. Appendix

15.1 Key API Endpoints (Quick Reference)

Device endpoints (Windows app uses these): - `POST /api/windows/devices/:id/heartbeat` - `GET /api/windows/devices/:id/pending-commands` - `POST /api/windows/devices/:id/upload-chunk` - `POST /api/windows/devices/:id/finalize-class` - `GET /api/windows/licenses/:key/validate`

Admin endpoints (admin portal uses these): - `GET /api/windows/devices` - `GET /api/windows/recordings` - `POST /api/windows/devices/:id/command` - `POST /api/windows/devices/:id/live-watch/start` - `POST /api/windows/licenses/issue` - `GET /api/windows/licenses`

15.2 Reference Documents

- LectureLens Android Edition architecture (existing system)
- LiveKit cloud documentation: https://docs.livekit.io
- Azure Blob Storage SDK: https://learn.microsoft.com/azure/storage/blobs
- OBS WebSocket Protocol: https://github.com/obsproject/obs-websocket
- Windows IoT Enterprise LTSC: https://learn.microsoft.com/windows/iot

15.3 Glossary

Term	Definition
Hybrid architecture	Recording happens locally, with cloud upload in background; live-watch is on-demand cloud streaming separate from recording
LiveKit Egress	Cloud server that records LiveKit room sessions to MP4
Dante	Audio-over-IP protocol from Audinate, industry standard for professional audio
DVS	Dante Virtual Soundcard — Windows software that exposes Dante audio as standard Windows audio device
MSIX	Modern Windows app packaging format; supports auto-update and rollback
DPAPI	Windows Data Protection API; encrypts secrets bound to a specific machine
TPM 2.0	Trusted Platform Module; hardware security chip in modern PCs
Quick Sync	Intel's hardware video encoder built into i3+ CPUs
NVENC	NVIDIA's hardware video encoder built into modern GPUs
LTSC	Long-Term Servicing Channel; Windows variant with 10-year support and no feature updates

---

End of document.

Prepared by: D&R AI Solutions Engineering **Review status:** Draft for approval **Next step:** Customer review + sign-off → begin Phase 1